

Author

- Name: Adarsh Kiran Varma
- Roll No.: 24F1000922
- 24f1000922@ds.study.iitm.ac.in
- I am a dual degree student pursuing a B.Tech in Metallurgical and Materials Engineering from VNIT Nagpur, along with a B.S. in Data Science. I am eager to apply AI and machine learning techniques to the field of materials science.

Description

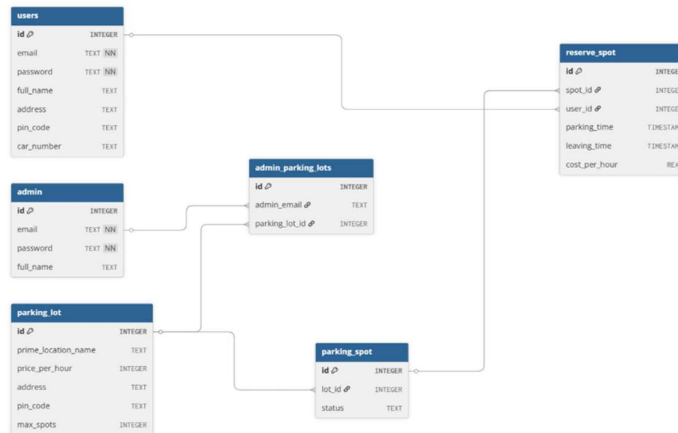
- In my understanding the project is about how to handle the SQLite data base properly with the effective and interactive front end. Also, the use of Flask to create proper workflow of the complete project.
- I haven't used AI/LLM for backend logics. I used AI for only good-looking theme/FrontEnd. Following is the chart for about usage of AI/LLM:

Purpose	Category	Tool	Usage %
Interactivity	Frontend	ChatGPT	5%
Styling and layout	Frontend	Gemini	~10%
HTML page generation	Backend	Gemini/ChatGPT	~10%

Technologies used

- **Flask:** Core framework to build the web application, handle routing, and manage HTTP requests/responses.
- **render_template:** Renders HTML templates with dynamic content.
- **request:** Handles incoming request data (GET/POST).
- **redirect & url_for:** Redirects users and generates dynamic URLs.
- **session:** Manages user sessions across pages (e.g., login state).
- **flash:** Displays one-time messages to users (e.g., login success/failure).
- **jsonify:** Returns JSON responses from routes (used for APIs or AJAX responses).
- **Blueprint:** Organizes the application into modular components (admin, user), improving structure and scalability.
- **sqlite3:** Lightweight database engine for storing and retrieving application data.

DB Schema Design



- This application uses an SQLite database named parking.db that supports all user management, parking spot availability, reservations, and admin operations. The database contains 6 tables; each designed to ensure normalized data and maintain referential integrity.

- Table: users :

- 'id': Unique user ID
- Email: User's email address
- Password: User's password
- full_name: Full name of the user
- address: Residential address
- pin_code: Area postal code
- car_number: Registered car number

Table: admin :

- id: Unique admin ID
- email: Admin email
- password: Admin password
- full_name: Admin full name

Table: parking_lot :

- 'id': Unique parking lot ID
- prime_location_name: Key location or landmark name
- price_per_hour: Hourly parking rate
- address: Residential address
- pin_code: Area postal code
- max_spots: Total number of spots in the lot

Table: parking_spot :

- id: Unique spot ID
- lot_id: Lot this spot belongs to
- status: 'O' = Occupied, 'A' = Available

Table: reserve_spot :

- id: Reservation ID
- spot_id: Reserved parking spot ID

- user_id: User who made the reservation
- parking_time: Start time of reservation
- leaving_time: End time of reservation
- cost_per_hour: Parking rate at the time of reservation

Table: admin_parking_lots:

- id: ID
 - admin_email: Admin responsible for this parking lot
 - parking_lot_id: Mapped parking lot
- Users and admins are stored in separate tables to simplify role-based operations and access control.
 - Parking lots, spots, and reservations are separated to avoid redundancy and maintain scalability.
 - status field in parking_spot ensures real-time tracking of spot availability.
 - **Scalability:** Use of admin_parking_lots as a mapping table enables flexible many-to-many relationships between admins and lots.
 - **Referential Integrity:** Foreign key constraints enforce data consistency across tables (e.g., reservations can't exist for non-existent users or spots).
 - **Efficiency in Queries:** Indexing via primary keys and uniqueness constraints helps optimize database queries.

Architecture and Features

The project is structured into four main directories—controllers, instance, static, and templates—along with a central app.py file that serves as the entry point for the Flask application. The app.py file handles user authentication processes such as login, signup, and logout, and routes requests to the appropriate controller files using Flask Blueprints. The controllers folder contains separate Python scripts for different functionalities: admin.py manages routes related to administrative tasks (e.g., dashboard, summary), while user.py handles user-specific routes. The control.py script provides utility functions for interacting with the SQLite database, including data retrieval, insertion, and modification. The parking_db.py script initializes the database by creating tables such as users, admin, and parking_lot, and also sets up the default admin account. The instance folder stores the SQLite database file (parking.db). The static folder contains CSS files used for styling the web pages, while the templates directory holds all HTML files, organized into separate subfolders for admin and user interfaces.

The project includes a range of core and additional features for both **Admin** and **User** roles, implemented using Flask routes, HTML forms, and SQLite database operations. Below is a detailed description of the functionality provided:

Admin Features:

1. Admin Authentication:

The admin can log in using credentials (email and password) that are generated during database initialization via the parking_db.py script.

2. Add Parking Lots:

Admins can add new parking lots using an HTML form. The input from the form is processed and stored in the SQLite database using SQL queries.

3. User Overview:

A “Summary” tab provides the admin with a complete list of all registered users.

4. Current Parking Status:

Admins can view the real-time status of any parking spot, including details of the user currently parked there, by clicking on the respective spot.

5. Parking Lot Summary and Analytics:

Each parking lot has a detailed summary section that displays historical usage data, including traffic over the last 7 days, total revenue generated in the last 7, 30, and 365 days, and other relevant insights.

6. Logout:

Admins can securely log out of their session using the logout button.

User Features:

1. Slot Booking:

Users can view all available parking lots and book a slot using a user-friendly interface.

2. My Parking Section:

Booked slots are visible in the “My Parking” section, where users can also perform the “Unpark” action once they leave the spot.

3. Parking History:

A dedicated “Summary” section allows users to view their complete parking history, including timestamps and slot details.

Video

<https://drive.google.com/file/d/1THRuDdYNikQVVCBAUBdzVdOA8Td05P/view?usp=sharing>